

Tribhuvan University

Faculty of Humanities and Social Science

A Project Report

On

AI Career Prediction System With Integrated Job Portal

Submitted To

Department of Computer Application

Xavier International College

Kathmandu

In partial fulfillment of the requirements for the Bachelors in Computer Application

Submitted By

Anjali Shrestha (6-2-546-22-2021)

Bishal Rajbanshi (6-2-546-26-2021)

Under Supervision of

Ankit Nepal

September,2026



Tribhuvan University

Faculty of Humanities and Social Sciences

Xavier International College

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by Anjali Shrestha and Bishal Rajbanshi entitled **"AI Career Prediction System with Integrated Job Portal"** in partial fulfilment of the requirements of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

_____

Ankit Nepal

SUPERVISOR

Xavier International College

Tusal,Boudha-Kathmandu



Tribhuvan University

Faculty of Humanities and Social Sciences

Xavier International College

LETTER OF APPROVAL

This is to certify that this project prepared by **Anjali Shrestha (54602066) and Bishal Rajbanshi (54602071)** entitled **"AI Career Prediction System with Integrated Job Portal"** in partial fulfilment of the requirements for the degree of Bachelor of Computer Application has been evaluated in our opinion. It is satisfactory in scope and quality as a project for the required degree.

_____ Ankit Nepal **Supervisor** Xavier International College Tusal, Kathmandu	_____ Rajesh Nepal **Co-Ordinator** Xavier International College Tusal, Kathmandu
_____ **Internal Examiner**	_____ **External Examiner**

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Tribhuvan University, Faculty of Humanities and Social Sciences**, and **Xavier International College** for providing me with the opportunity to undertake this project as a part of the requirements for the degree of Bachelor of Computer Application. This project has provided me with an opportunity to apply the theoretical knowledge and practical skills gained throughout my academic journey to the development of a real-world software application.

I am deeply grateful to my project supervisor, **Mr. Ankit Nepal**, for his valuable guidance, continuous support, constructive suggestions, and encouragement throughout the development of **"AI Career Prediction System with Integrated Job Portal"**. His guidance and feedback greatly helped me in understanding the project requirements, improving the system design, solving technical challenges, and completing the project successfully.

I would also like to express my sincere thanks to **Mr. Rajesh Nepal, Co-Ordinator**, the **Department of Computer Application at Xavier International College**, and all the respected faculty members for their academic guidance, valuable suggestions, and continuous encouragement throughout my BCA studies. Their knowledge and support have played an important role in developing my technical and academic abilities.

I am grateful to my family members and friends for their unwavering support, motivation, encouragement, valuable suggestions, discussions, testing support, and patience during the development of this project. Their encouragement inspired me to overcome challenges and stay committed to achieving my goals.

Finally, I would like to acknowledge the availability of online resources, documentation, resource providers, open-source contributors, writers, and community support for the technologies used in this project, which greatly aided in problem-solving and learning.

Anjali Shrestha

Bishal Rajbanshi

Xavier International College

ABSTRACT

Many students find it hard to know which career fits their skills, and companies get too many resumes to check one by one. This project builds a website that helps both sides.

A job seeker uploads a resume as a PDF. The website reads the text, finds the skills in it, and suggests a good career path for that person. It also gives a score for the resume, lists any missing skills, and recommends courses and interview questions. If the user also pastes a job advertisement, the website shows a percentage match for that job.

The website also works as a simple job board. Companies that are checked and approved by an admin can post open jobs. Job seekers can search and apply for these jobs. Every application automatically gets a match score, so companies can see the best-fitting candidates first and move them through hiring stages.

The system is built from three parts working together: a React website for the user, a Laravel server that manages accounts and data, and a separate Python (FastAPI) service that reads resumes and predicts careers. It was built step by step in small stages and checked using test cases. The career model was trained on a mix of public data and about 13,000 real, anonymised resumes, and reaches roughly 86% accuracy on a held-out test set. Future updates should make the skill list even bigger and the matching even smarter.

Keywords: career prediction, resume reading, job portal, machine learning, web application.

TABLE OF CONTENTS

ACKNOWLEDGEMENT ..... **iii**
ABSTRACT ..... **iv**
LIST OF ABBREVIATIONS ..... **vii**
LIST OF FIGURES ..... **viii**
LIST OF TABLES ..... **ix**
CHAPTER 1: INTRODUCTION **1**
1.1 Introduction 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope and Limitation 2
1.4.1 Scope 2
1.4.2 Limitations 2
1.5 Development Methodology 3
1.6 Report Organization 4
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW **5**
2.1 Background Study 5
2.2 Literature Review 6
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN **7**
3.1 System Analysis 7
3.1.1 Requirement Analysis 7
3.1.2 Feasibility Analysis 10
3.1.3 Object Modelling using Class and Object Diagrams 11
3.1.4 Dynamic Modelling using State and Sequence Diagrams 12
3.1.5 Process Modelling using Activity Diagrams 13
3.2 System Design 14
3.2.1 Architecture and Database Design 14
3.2.2 Refined Diagrams 16
3.2.3 Component Diagram 18
3.2.4 Deployment Diagram 18
3.3 Algorithm Details 19
CHAPTER 4: IMPLEMENTATION AND TESTING **21**

4.1 Implementation.....21
4.1.1 Tools Used.....21
4.1.2 Implementation Details of Modules.....21
4.2 Testing.....22
4.2.1 Test Cases for Unit Testing.....22
4.2.2 Test Cases for System Testing.....23
4.3 Result Analysis.....23
CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS.....25
5.1 Conclusion.....25
5.2 Future Recommendations.....25
5.2.1 Smarter AI.....25
5.2.2 Safety and Reliability.....25
5.2.3 Better Engineering.....26
5.2.4 New Features.....26
REFERENCES.....27
BIBLIOGRAPHY.....29
APPENDIX A: SCREENSHOTS.....30

LIST OF ABBREVIATIONS

Abbreviation	Full form
AI	Artificial Intelligence
API	Application Programming Interface
ATS	Applicant Tracking System
BCA	Bachelor of Computer Applications
CASE	Computer-Aided Software Engineering
CSV	Comma-Separated Values
JSON	JavaScript Object Notation
JWT	JSON Web Token
ML	Machine Learning
NER	Named Entity Recognition
NLP	Natural Language Processing
ORM	Object-Relational Mapping
PDF	Portable Document Format
REST	Representational State Transfer
RSS	Really Simple Syndication
SBERT	Sentence-BERT
SPA	Single-Page Application
TF-IDF	Term Frequency – Inverse Document Frequency
UML	Unified Modeling Language

LIST OF FIGURES

Figure 1: Diagram of the Iterative and Incremental Development Model.....3
Figure 2: Use Case Diagram of the System.....9
Figure 3: Gantt Chart of the Project Schedule.....11
Figure 4: Class Diagram of the System.....11
Figure 5: Object Diagram of the System.....12
Figure 6: State Diagram of the System.....12
Figure 7: Sequence Diagram of the System.....13
Figure 8: Activity Diagram of the System.....14
Figure 9: System Architecture.....15
Figure 10: Database Schema (relational).....16
Figure 11: Refined Class Diagram of the System.....17
Figure 12: Class Diagram of the Controller and Service Layer.....17
Figure 13: Refined State Diagram of the System.....17
Figure 14: Component Diagram of the System.....18
Figure 15: Deployment Diagram of the System.....19
Figure 16: Login/Signup Page and Job Seeker Dashboard.....30
Figure 17: Resume Upload Page and Analysis Result Page.....30
Figure 18: Skill Gaps and Analyze Page.....31
Figure 19: Job Details/Apply Page and My Applications Page.....31
Figure 20: Employer Job Posting Page and Admin Moderation Page.....32

LIST OF TABLES

Table 1: Unit Testing Results.....22
Table 2: System Testing Results.....23

CHAPTER 1: INTRODUCTION

1.1 Introduction

Picking a career and finding a job is hard for students and new graduates. Most students know what they studied, but not what skills companies actually want or which skills they are missing. Companies, on the other hand, get many resumes and spend a lot of time checking each one by hand.

This project, the AI Career Prediction System with Job Portal (CareerAI), tries to fix both problems. A job seeker uploads a resume as a PDF. The system reads the text, picks out the skills, and suggests the best-fitting career out of 25 possible careers. It also gives a resume score, lists the missing skills, and suggests jobs, courses, and interview questions. If the user pastes a job description too, the system shows a percentage match.

The website also works as a simple job board. Companies can create a profile and post job openings, but an admin has to approve them first. Job seekers can search for jobs and apply. Every application automatically gets a match score, so the company can see the best-fit candidates first and move them through hiring stages such as "shortlisted," "interview," or "selected."

Behind the scenes, the system has three parts that work together: a React website that the user sees, a Laravel server that handles logins, files, and the database, and a separate Python service (FastAPI) that does the AI work, such as reading skills and guessing careers. Keeping the AI work in its own service makes it easier to build, test, and improve without breaking the rest of the site.

1.2 Problem Statement

- Students often don’t know which skills they are missing for the career they want.
- Companies spend too much time reading resumes one by one, especially small companies.
- Career advice tools and job-posting sites are usually separate, so a student has to use several websites.
- Resumes rarely come with feedback that explains why they are weak for a certain job.
- Small companies often can’t afford large, paid hiring software.

1.3 Objectives

The main goal of this project is to build a website that combines AI-based career guidance with a job portal. The specific goals are:

- Read PDF resumes and pull out the skills, name, email, and companies mentioned.
- Suggest a good career, out of 25 possible careers, using both simple rules and a trained machine-learning model.
- Give a resume score, a job-match percentage, and a list of missing skills.
- Recommend jobs, courses, and interview questions.
- Let companies post jobs (with admin approval) and let job seekers apply.
- Automatically rank job applicants by how well they match a job.
- Show a simple dashboard with charts and a history of past resumes.

1.4 Scope and Limitation

1.4.1 Scope

- Three types of users are supported: job seeker, employer, and admin, each with their own permissions.
- Job seekers can upload PDF resumes (up to 2 MB), see their results, search and save jobs, and apply.
- Employers can manage a company profile, post and edit jobs, and see ranked applicants.
- Admins can approve employers and jobs, manage users, and refresh job listings from other websites.
- The whole system works inside a normal web browser.

1.4.2 Limitations

- Only readable (text-based) PDFs work; scanned or photographed resumes don’t, since there is no picture-to-text (OCR) feature yet.
- The skill list currently covers a little over 120 skills, so a very rare or niche skill may not be recognised.
- The career model covers 25 common careers; a career outside this list cannot be suggested.

- Matching is based on comparing words, not real meaning, so similar words (synonyms) can be missed.
- The AI's suggestions are meant to help, not replace, a real decision made by the person or the recruiter.
- Features such as password reset, in-app messaging, video interviews, and online payment are not included yet.
- Some job listings come from other websites, so they depend on those websites working correctly.

1.5 Development Methodology

The project was built using the iterative and incremental method [1]. Instead of building everything at once, the work was split into five smaller stages. Each stage added new features, was tested, and built on top of the last stage (Figure 1). This way, both the AI part and the website part could be tested early, and new ideas — like the job portal — could be added later without starting over.

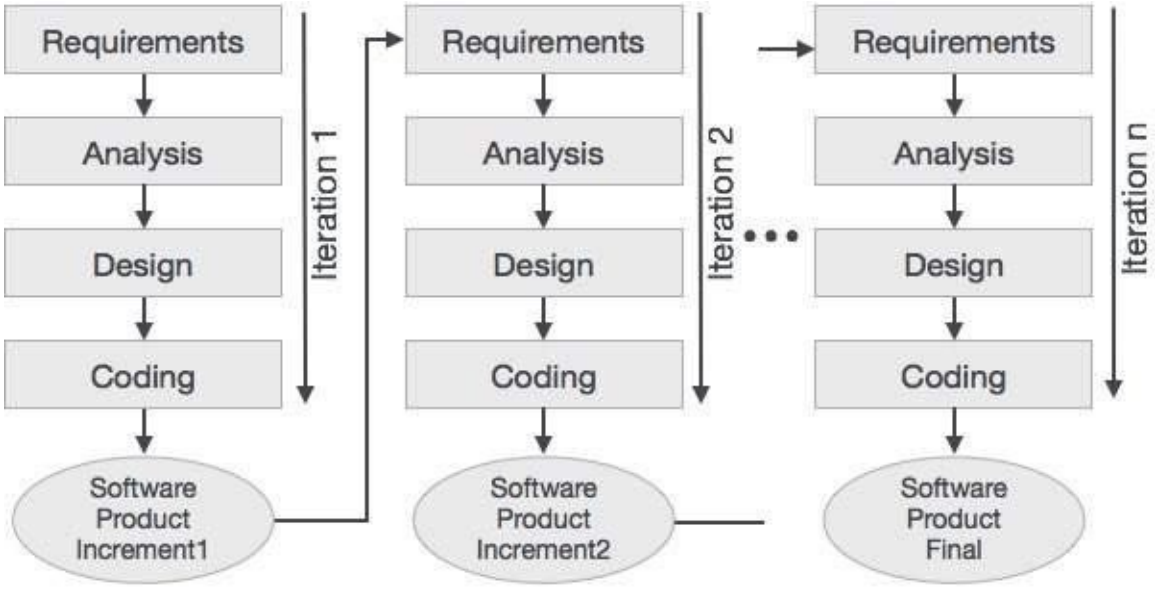


Figure 1: Diagram of the Iterative and Incremental Development Model

Stage 1 added logins, resume upload, and reading text from PDFs. Stage 2 added the AI service that finds skills and guesses careers. Stage 3 added the dashboard and charts. Stage 4 added job search and saved jobs. Stage 5 turned the system into a full job portal, with employers, job posting, admin approval, applications, and applicant ranking.

1.6 Report Organization

This report has five chapters. Chapter 1 introduces the project, the problem, the goals, and the plan used to build it. Chapter 2 explains background ideas and looks at similar tools. Chapter 3 covers how the system was analysed and designed. Chapter 4 explains how it was built and tested. Chapter 5 gives the conclusion and ideas for future work. After that come the references and appendices with screenshots and source code.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

The project uses several key techniques to process resumes, identify skills, predict careers, and provide relevant job recommendations. For extracting text from PDF resumes, the system uses **Poppler [2]**, which converts text-based PDF content into plain text. This works effectively for digitally created resumes, although scanned image-based resumes are not currently supported because OCR is not included.

For resume analysis, **spaCy [3]** is used to identify important information such as candidate names and company names, while simple pattern matching is used to detect email addresses. The system also maintains a database of more than 120 predefined skills, including common variations such as “React” and “React.js.” These skills are matched against resume content using exact word matching to avoid incorrect matches, although the approach is limited to skills included in the predefined list.

To compare resumes with job descriptions, the system applies **TF-IDF [4]** to convert both texts into numerical representations and uses **cosine similarity** to calculate their level of similarity. This generates the job match percentage presented to users. In addition, a machine-learning model developed using **scikit-learn [5]** predicts suitable career fields based on the language and information contained in a resume. The model is trained using a combination of public career data, manually prepared career-skill mappings, and approximately 13,000 anonymized resumes, providing a broader range of training examples.

The system combines rule-based methods with machine learning to improve the overall analysis. Once a career is predicted, the system compares its required skills with the skills identified in the resume. The unmatched skills are presented as **skill gaps**, while the percentage of matched skills contributes to the **resume score**. This helps users understand both their current strengths and areas that may require improvement.

The integrated job portal allows employers to post vacancies, while administrators review and approve listings before they become publicly available. This moderation process helps maintain the quality of job listings. The system also uses **role-based access**

control, allowing job seekers, employers, and administrators to access only the functions relevant to their roles.

Technically, the system follows a **three-service architecture** consisting of a React [6] frontend, Laravel [7] backend, and FastAPI [8] AI service. These components communicate through APIs, while **JSON Web Tokens (JWT) [9]** are used for secure authentication. This separation keeps the user interface, core application logic, and AI processing organized and independently manageable.

2.2 Literature Review

Existing job platforms such as **LinkedIn [10]**, **Indeed [11]**, and Nepal-based **Merojob [12]** allow employers to post vacancies and job seekers to search and apply for jobs. However, they generally provide limited guidance on the specific skills users should develop for a particular career path. Tools such as **Jobscan [13]** provide resume-to-job matching and generate a match score, but their primary focus is resume evaluation rather than providing a complete career and recruitment platform.

Research has also explored advanced AI models such as **BERT [14]** for matching resumes and job descriptions based on semantic meaning rather than simple word matching. Although these models can provide more sophisticated results, they require greater computational resources. Therefore, the current system uses the comparatively lightweight **TF-IDF** approach, while more advanced models can be considered for future improvement. Public skill frameworks such as **ESCO [15]** and **O*NET [16]** could also be integrated later to expand the system's skill database.

Overall, existing solutions generally focus either on **job searching** or **resume evaluation**, with limited integration of career guidance, skill-gap analysis, and applicant ranking. This project aims to combine these functions into a single platform, helping job seekers understand their career and skill requirements while supporting employers in managing and evaluating applicants.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

This chapter looks at what the system needs to do, checks whether it can realistically be built, and then models it using simple diagrams, following standard UML modelling practice [17]. The early diagrams describe the problem from the user's point of view, and the later ones show the actual classes, parts, and setup that were used to build the system.

3.1 System Analysis

The system has three kinds of users. A job seeker uploads a resume, gets AI feedback, and searches and applies for jobs. An employer manages a company profile, posts jobs, and reviews applicants. An admin approves employers and jobs, manages users, and refreshes job listings pulled in from other sites.

3.1.1 Requirement Analysis

What the system needs to do, grouped by area:

Accounts and access

- Let a visitor sign up as a job seeker or employer, and let users log in and out securely.
- Show or hide features depending on whether the user is a job seeker, employer, or admin.
- Block login for an account the admin has deactivated.

Resume analysis

- Accept one PDF resume (up to 2 MB) and, optionally, a job description; or let a user paste resume text directly for a quick check.
- Pull text from the PDF and show a clear error if nothing can be read.
- Find the candidate's name, email, companies, skills, and, where possible, their location.
- Suggest the top matching careers, out of 25, using both simple rules and machine learning.
- Work out a resume score, a job-match percentage, and a list of missing skills.
- Suggest jobs, courses, and interview questions.

- Save each analysis so the user can view or delete it later, and keep working even if the AI service is briefly unavailable.
- Show a dashboard with total analyses, average score, top career, and simple charts.

Jobs

- Let users search and filter job listings, including ones imported from other sites.
- Rank recommended jobs using the user's latest skills and predicted career.
- Let job seekers save and unsave jobs.
- Let the admin refresh job listings from outside sources.

Employer and hiring

- Let an employer create and edit a company profile, once approved by the admin.
- Let an employer post a job with details such as title, skills, location, and deadline.
- Let an employer edit or close their own jobs; editing an already-approved job sends it back for approval.
- Show applicants for a job ranked by match score, and let the employer update their status.
- Let a job seeker apply to a job once, using an analysed resume and an optional cover letter.
- Let a job seeker track application status or withdraw an application.

Administration

- Let the admin approve or reject employer accounts and job postings.
- Let the admin view, search, and manage users, including activating or deactivating an account.
- Show the admin a dashboard of overall usage and a summary of the external job feed.

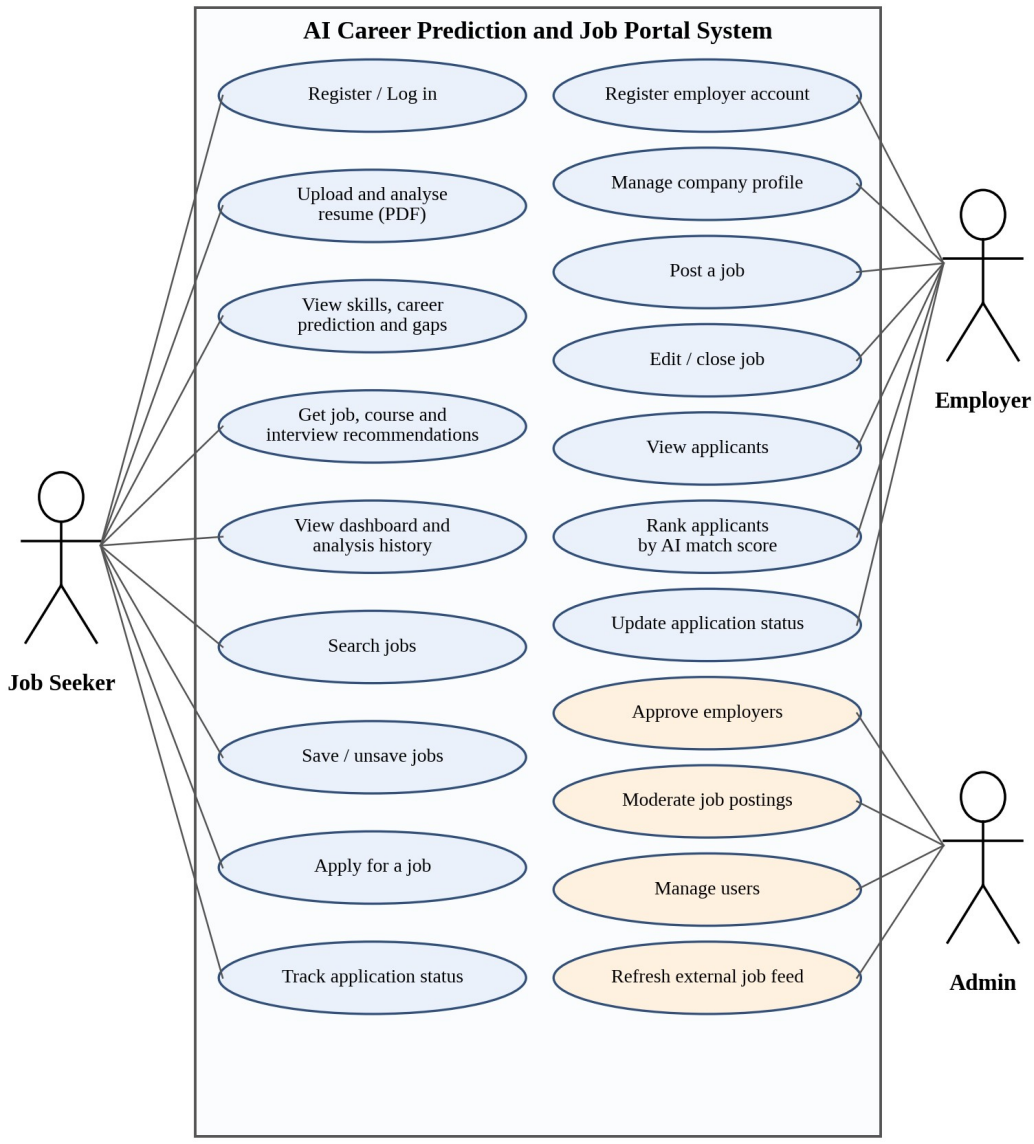


Figure 2: Use Case Diagram of the System

A few of the most important actions are explained below.

Register and log in: A new user fills in their name, email, password, and role, and the system creates their account. When they log in, the server checks the password and gives back a secure pass (JWT) that lets them use the site without logging in again on every page. This pass refreshes itself quietly in the background, and wrong passwords, a deactivated account, or an email already in use all show a clear error message.

Upload and analyse a resume: A job seeker picks a PDF and, optionally, pastes a job description, then submits it. The server saves the file, pulls out the text, and sends it to the AI service, which returns the skills, top career guesses, score, missing skills, location, and suggestions. If the file is invalid or empty, an error appears and nothing is saved.

Search and save jobs: A job seeker types a keyword or picks a filter to see matching jobs (both posted and imported ones), switches to a "Recommended" tab to see jobs ranked for their own skills, and can save any job to look at later.

Apply for a job: A job seeker opens an open job, picks one of their analysed resumes, and applies (only once per job). The system works out a match score for that application in the background, just after the request is sent.

Post a job: An employer fills in a job form. The job is saved as "pending" until the admin approves it, after which it becomes visible to job seekers. If it is rejected, the employer can fix it and send it again.

Review applicants: An employer opens a job's applicant list, ranked by match score, and updates each applicant's status through the stages applied, shortlisted, interview, and selected or rejected.

3.1.2 Feasibility Analysis

The project was checked for feasibility in three simple ways.

Technical feasibility: All the tools used — React [6], Laravel [7], FastAPI [8], spaCy [3], and scikit-learn [5] — are free, well documented, and run on a normal laptop, so the technology was realistic for a student project.

Economic feasibility: Since free and open-source tools were used, the main cost was the students' time, with only a small cost for internet and possible hosting — no licence fees were needed.

Schedule feasibility: The project was planned over roughly sixteen weeks, split into the five stages described in Section 1.5, shown in the Gantt chart in Figure 3. Requirement analysis and design took the first few weeks, most of the coding happened in the middle weeks, and the last weeks were used for the job portal features, testing, and this report.

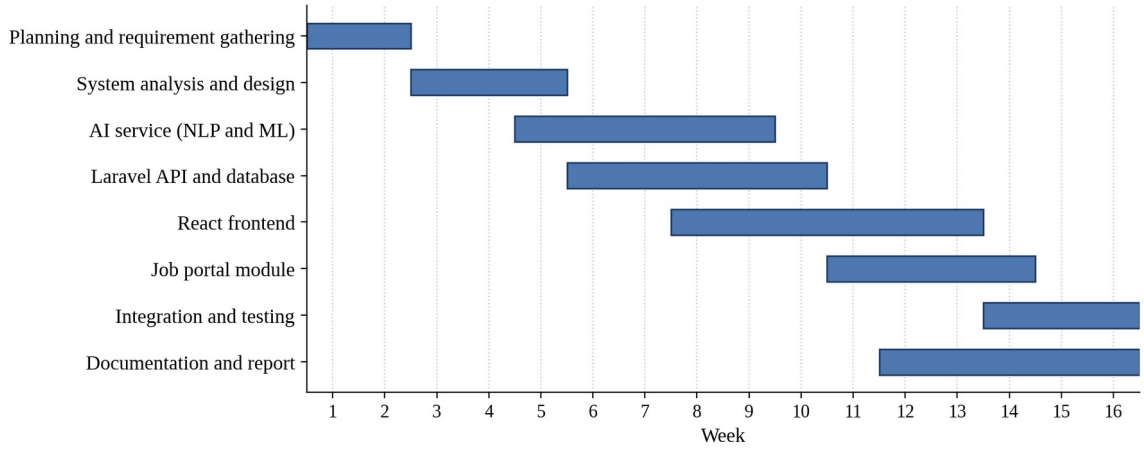


Figure 3: Gantt Chart of the Project Schedule

3.1.3 Object Modelling using Class and Object Diagrams

Figure 4 shows the main classes of the system, such as User, CompanyProfile, Resume, JobListing, and JobApplication, along with their basic details and how they relate to each other. Figure 5 shows an object diagram — a snapshot with example data — to make the class diagram easier to follow.

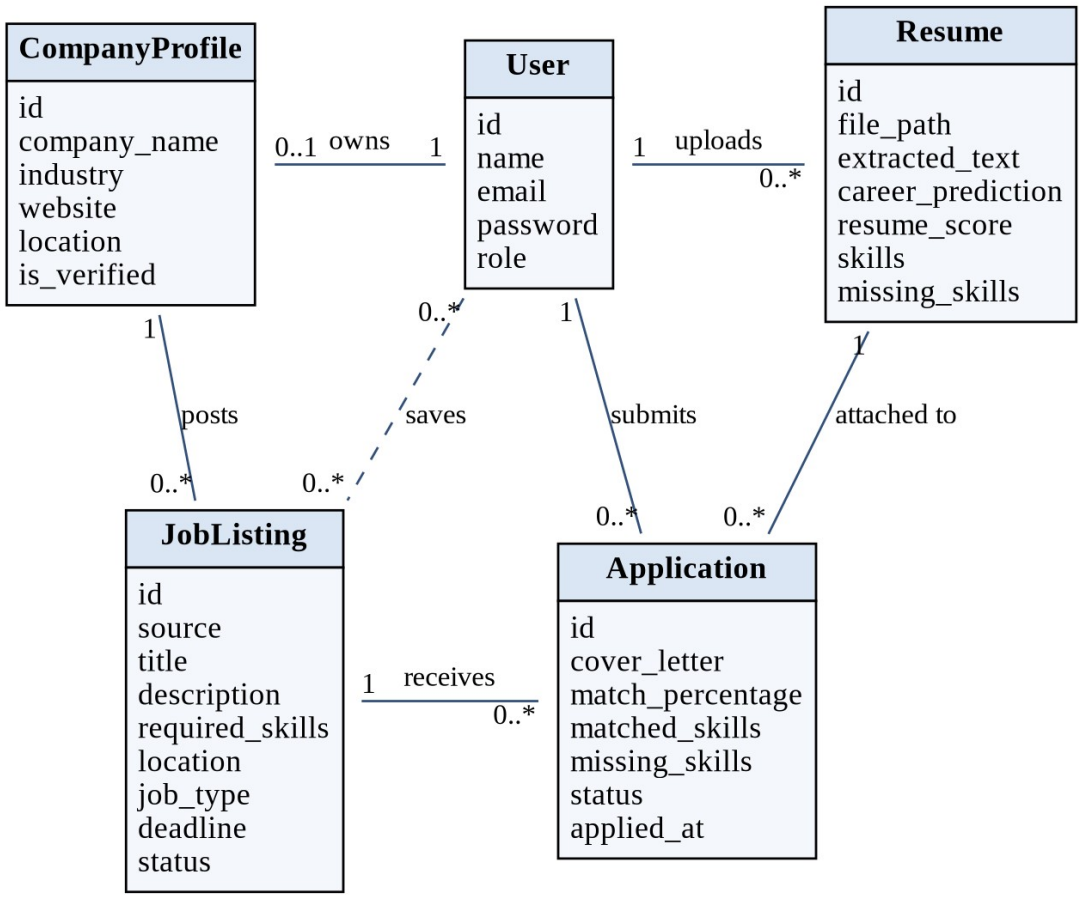


Figure 4: Class Diagram of the System

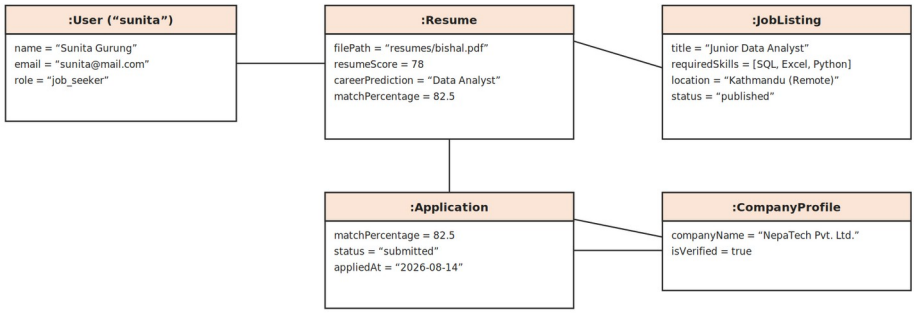


Figure 5: Object Diagram of the System

3.1.4 Dynamic Modelling using State and Sequence Diagrams

Figure 6 shows how a job application or a job posting moves through different stages — for example, from "applied" to "shortlisted" to "selected," or from "pending" to "published" to "closed."

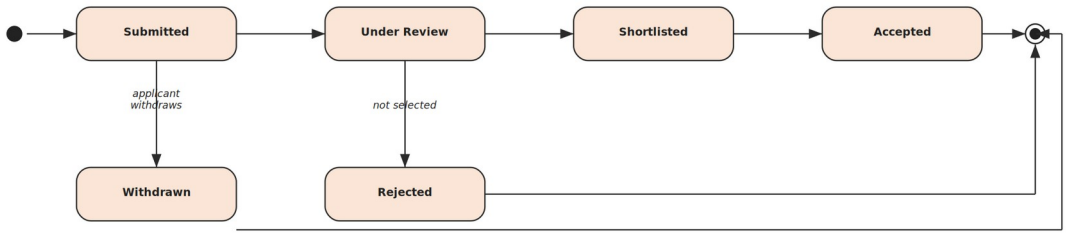


Figure 6: State Diagram of the System

Figure 7 shows, step by step, how the browser, server, and AI service talk to each other — first when a resume is analysed, and then when someone applies for a job and gets ranked.

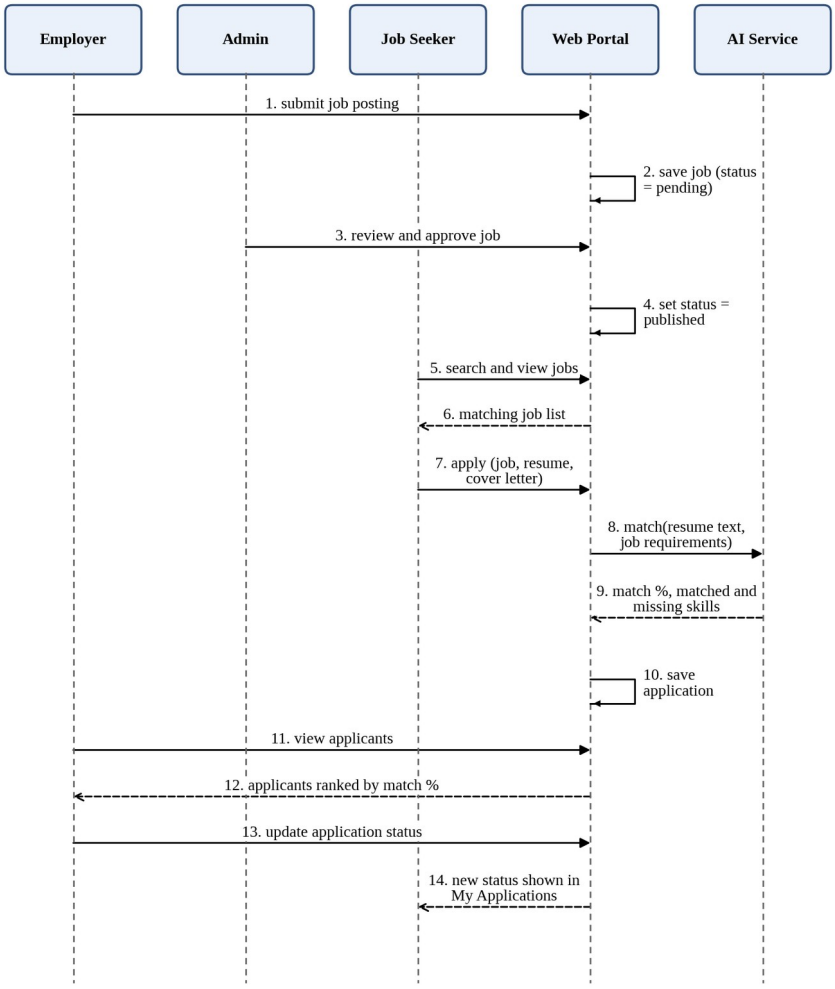


Figure 7: Sequence Diagram of the System

3.1.5 Process Modelling using Activity Diagrams

Figure 8 shows the flow of actions for both processes: resume analysis (including what happens if the file is invalid) and job posting and application, from the employer's post to the admin's approval to the job seeker's application.

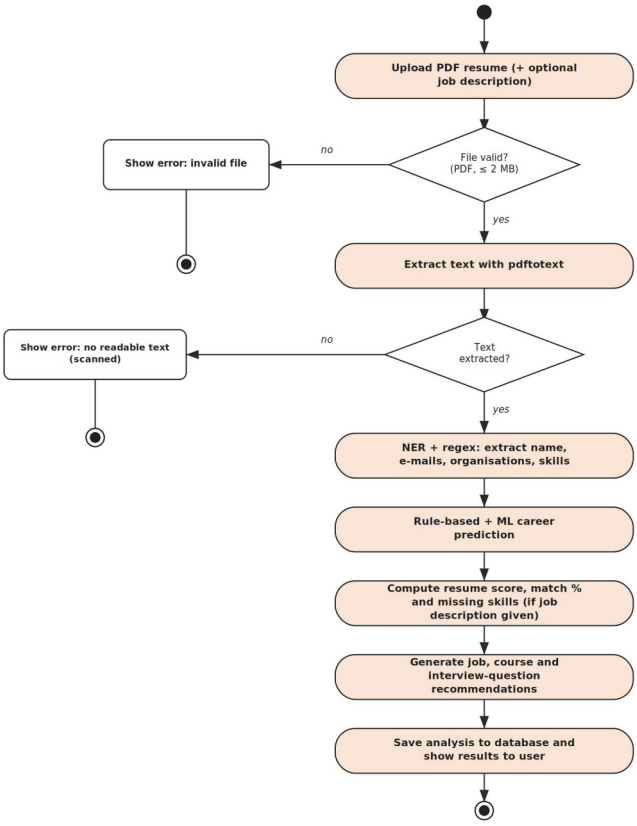


Figure 8: Activity Diagram of the System

3.2 System Design

3.2.1 Architecture and Database Design

The system is built in three layers. The React website only talks to the Laravel server, using a secure login pass (JWT) over a safe connection (HTTPS). The Laravel server is the only part that touches the database, the stored files, and the AI service, so regular users can never reach the AI service directly. The AI service is the only part that reads job listings from other websites.

The database keeps information in a few main tables. The users table stores login details and the role (job seeker, employer, or admin), plus whether an employer account is approved. The company_profiles table stores a company's details. The resumes table stores the uploaded file's location, the extracted text, and the full AI result as one saved

report. The job_listings table stores job details, whether it was posted on the site or imported, and its moderation status (pending, published, closed, or rejected). The job_applications table links a job seeker, a job, and a resume, and stores the match score, matched and missing skills, a rank, and a status. The saved_jobs table simply records which jobs a job seeker has saved.

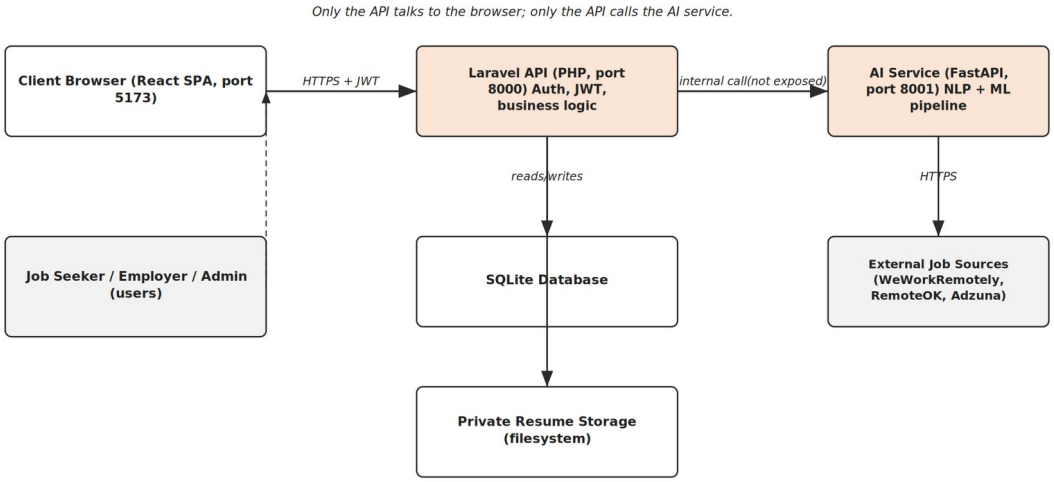


Figure 9: System Architecture

The relational schema is shown in Figure 10.

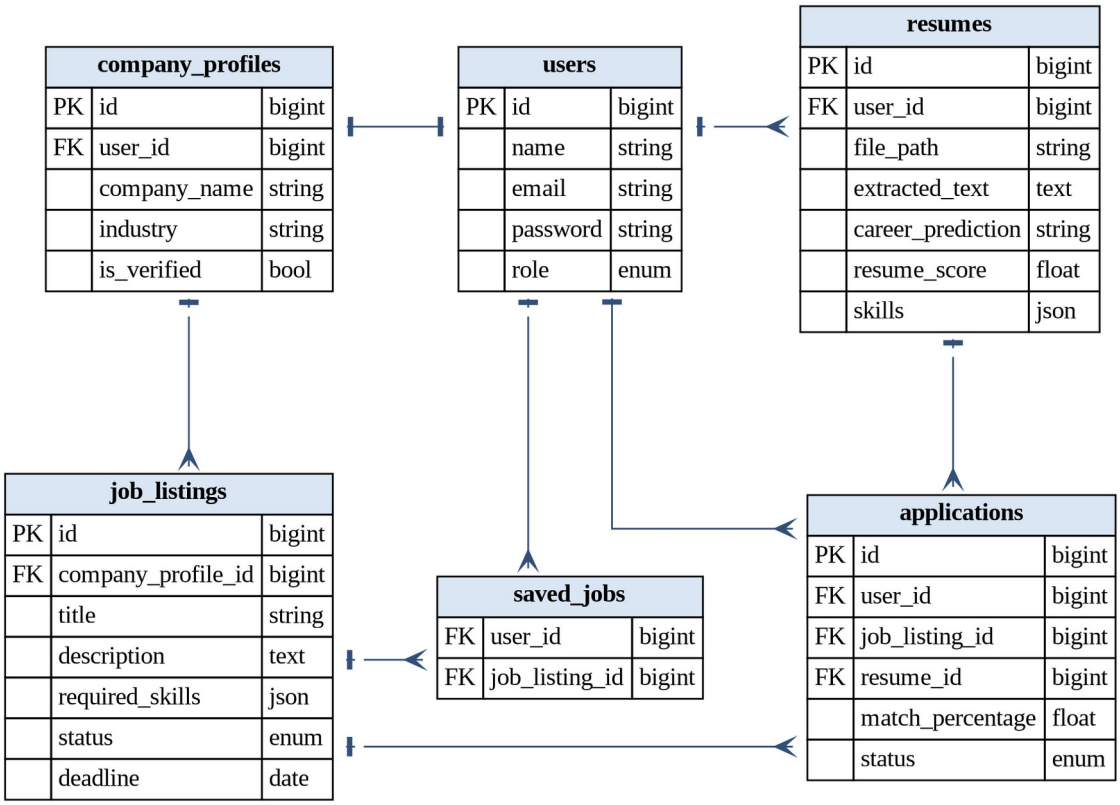


Figure 10: Database Schema (relational)

3.2.2 Refined Diagrams

After the first design pass, the class and object diagrams were refined by adding real data types and the actual controllers and services used in the code (Figure 11 and Figure 12). The stages for applications and jobs stayed the same, but a new, more detailed state diagram was added just for how one uploaded resume is processed, making clear that a failed analysis still returns a result to the user instead of an error (Figure 13).

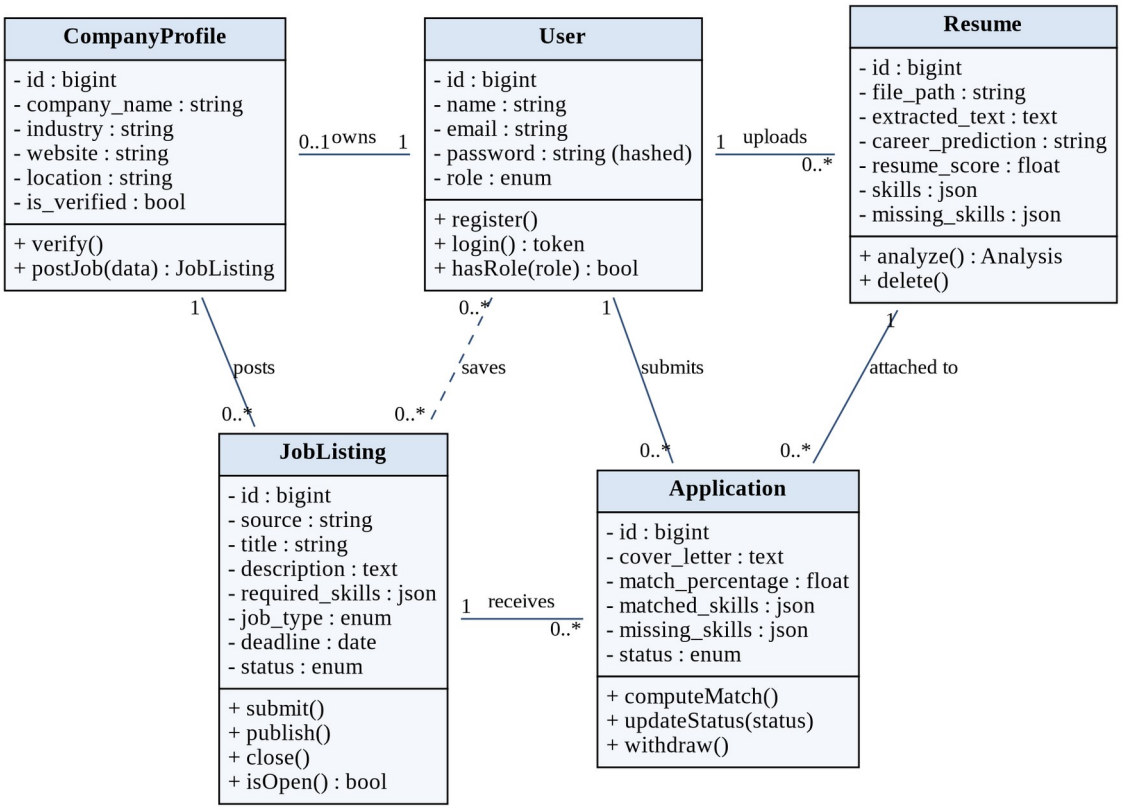


Figure 11: Refined Class Diagram of the System

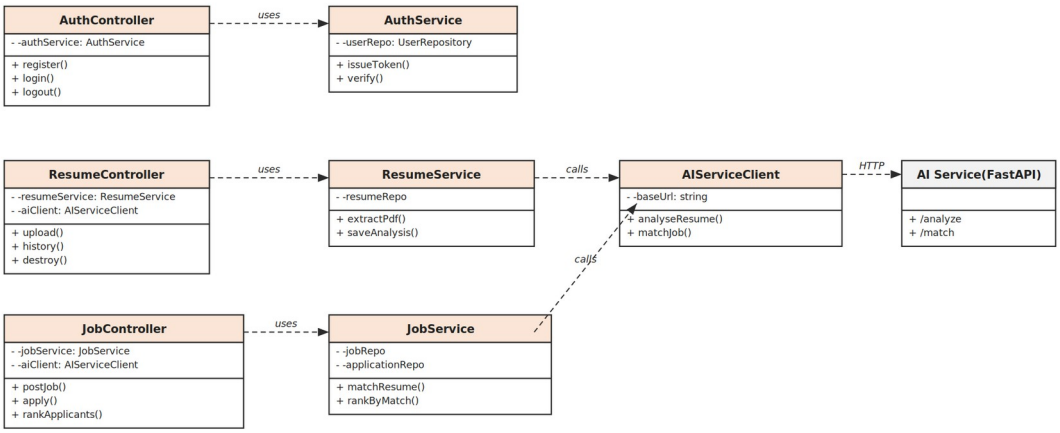


Figure 12: Class Diagram of the Controller and Service Layer

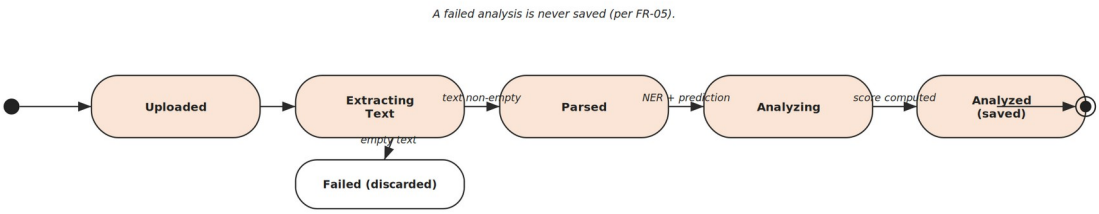


Figure 13: Refined State Diagram of the System

3.2.3 Component Diagram

Figure 14 shows how the different parts of the system depend on each other. Every request first passes through a login and role check before reaching the right part of the code, and only the resume, application, and job features talk to the AI service.

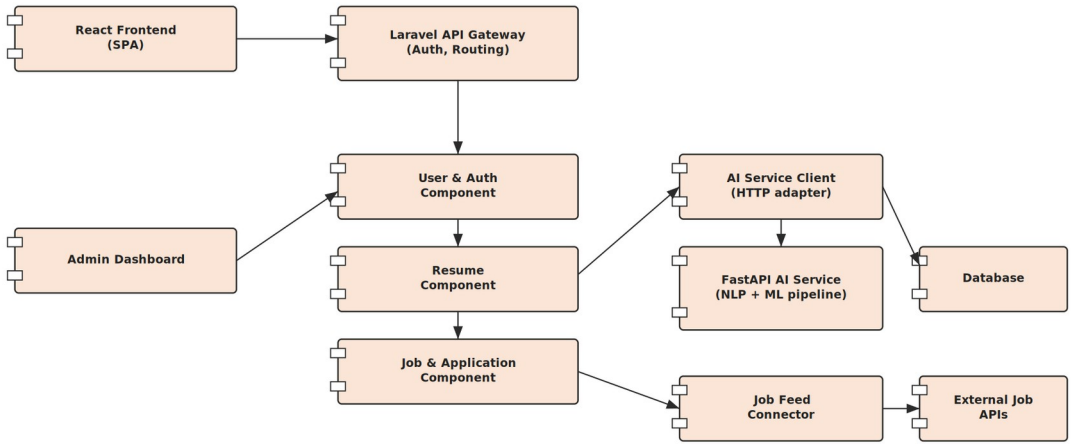


Figure 14: Component Diagram of the System

3.2.4 Deployment Diagram

Figure 15 shows how the three services are set up when running. They run as three separate programs, each on its own network port, which can sit on one computer or be spread across several. The server reads and writes the database and the private resume files, and the AI service is the only part that connects to the internet to read outside job listings.

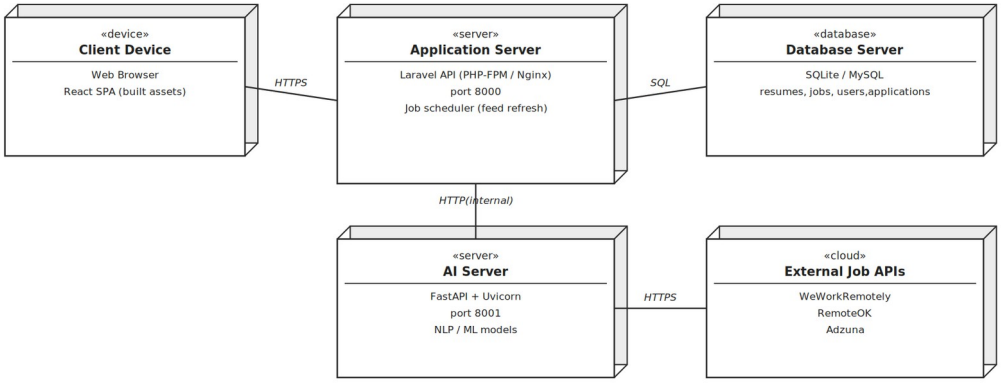


Figure 15: Deployment Diagram of the System

3.3 Algorithm Details

The system applies a sequence of rule-based and machine-learning techniques to analyze resumes, predict careers, match jobs, and rank applicants. First, the uploaded resume text is converted to lowercase and compared with a predefined skill database. Alternative spellings of each skill are also considered, and whole-word matching is used to avoid incorrect matches, such as detecting “Java” within “JavaScript.” Duplicate matches are removed to obtain the final set of identified skills.

For rule-based career prediction, the system evaluates each of the 25 predefined careers by comparing its required skills with the skills identified from the resume. The career with the highest skill-match ratio is selected as the rule-based prediction. The match ratio is calculated as:

Career Match Ratio = (Number of Matched Skills / Total Required Skills) × 100

As a second approach, the system uses a TF-IDF and Logistic Regression machine-learning model trained on resume-to-career data. TF-IDF converts resume text into numerical features, while Logistic Regression estimates the probability of each career category. The system returns the three careers with the highest predicted probabilities:

$$P(C_i \mid X) = \text{Predicted Probability of Career } C_i$$

where X represents the extracted resume features. The probabilities are presented as confidence percentages.

For job matching, the system compares the resume with each job description using TF-IDF and cosine similarity. The resulting similarity value is converted into a percentage:

$$\textbf{Job Match Score} = \textbf{Cosine Similarity}(R, J) \times 100$$

where R represents the resume and J represents the job description. If no job description is available, the job-match score is set to 0%.

The resume score is calculated by comparing the skills identified in the resume with the skills required for the predicted career:

$$\textbf{Resume Score} = (\textbf{Matched Required Skills} / \textbf{Total Required Career Skills}) \times 100$$

Skills required by the predicted career but not found in the resume are identified as missing skills, allowing users to understand the areas they may need to improve.

For job recommendations, each available job receives a score based on the user's existing skills and predicted career. Each matching required skill contributes 25 points, while a matching word between the job title and predicted career provides an additional 20 points:

$$\textbf{Job Recommendation Score} = 25M_s + 20B_c$$

where M_s is the number of matched skills and B_c is 1 when the job title matches the predicted career terms and 0 otherwise. Jobs with a score greater than zero are displayed and sorted in descending order.

For applicant ranking, the system combines skill overlap and resume-job text similarity. Skill matching contributes 70% of the score, while text similarity contributes 30%. An additional 15-point bonus is given when the applicant's predicted career matches the job title:

$$\textbf{Applicant Score} = 0.70S + 0.30T + 15B_c$$

where S is the skill-overlap percentage, T is the resume-job text similarity percentage, and B_c represents the career-title match indicator. Applicants are then ordered according to their scores, with applicants having equal scores receiving the same rank.

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Implementation

4.1.1 Tools Used

- Frontend: React, Vite, Tailwind CSS, and Recharts for the charts.
- Backend: PHP with Laravel, using tymon/jwt-auth for secure login passes (JWT).
- Reading PDFs: Poppler, used through Spatie's PdfToText package.
- Database: SQLite, accessed through Laravel's Eloquent tool.
- AI service: Python with FastAPI, scikit-learn for machine learning, and spaCy for reading names, companies, and locations.
- Training data: a public careers dataset, a hand-built skills table, and about 13,000 real, anonymised resumes.
- Testing tools: Laravel's own test runner and Python's pytest.

4.1.2 Implementation Details of Modules

Login and access control: New users pick a password with at least 8 characters, one capital letter, and one number, typed twice to confirm. Logging in returns a secure pass (JWT) that quietly renews itself before it expires. Each page checks the user's role, and a deactivated account is blocked at login with a clear error.

Resume upload and reading text: The server accepts a PDF up to 2 MB, limited to 10 uploads per minute per user to stop abuse, and pulls out the text using Poppler. If the AI service is briefly down, the upload still succeeds with an empty result and a note, instead of the whole request failing.

AI analysis: One request to the AI service returns the full picture: the name, email, and companies found; the skills found; the top three likely careers with a confidence percentage each, plus a simple rule-based guess as a cross-check; the resume score; the missing skills; the candidate's likely location; and matching jobs, courses, and interview questions. A second, lighter option lets a user paste resume text and a job description directly, for a quick match check without uploading a file.

Dashboard and analytics: The job seeker's dashboard shows simple stat cards (total analyses, average score, top career, saved jobs, applications), a career pie chart, and a score history bar chart, all drawn with Recharts.

Job search and outside listings: Job seekers can browse all jobs with filters, or switch to a "Recommended" tab ranked by their own skills and predicted career. Jobs from other sites are fetched by the AI service, checked for duplicates, and kept in a short-term cache so the page still works if an outside site is slow or down; an admin can force a refresh.

Job portal (employers, applications, moderation): A new employer account is not usable until the admin approves it. Once approved, the employer can manage a company profile and post jobs, which stay hidden until the admin approves them too; editing an approved job sends it back for approval. When a job seeker applies, the match score is worked out just after the request finishes, so the page does not need to wait. Employers see applicants ranked by this score and move each one through the stages applied, shortlisted, interview, and selected or rejected, and can download the applicant's resume.

Admin tools: The admin can approve or reject employers and jobs, search and manage users (including activating, deactivating, or promoting an account — though an admin cannot deactivate, delete, or demote their own account), and see counts of jobs coming from each outside source.

Website (frontend): The React site protects pages that need a login, shows a loading state while waiting for the server, and shows a clear message whenever something goes wrong.

4.2 Testing

Testing was done at two levels: unit tests checked individual functions and requests, and system tests checked complete user scenarios through the interface. The test cases are listed in Table 1 and Table 2. The Status column is filled in when each case is actually run.

4.2.1 Test Cases for Unit Testing

Table 1: Unit Testing Results

ID	What was tested	Input used	Expected result	Status
UT-01	Finding skills	Text with known skills	All known skills found, no repeats	Pass

4.2.2 Test Cases for System Testing

Table 2: System Testing Results

ID	Scenario	Steps taken	Expected result	Status
ST-01	Sign up and log in	Register, log in, open dashboard	Dashboard opens for the correct role	Pass
ST-02	Analyse a resume	Upload a valid PDF, no job description	Skills, careers, score, and gaps shown	Pass
ST-03	Analyse with a job	Upload a PDF and paste a job description	Match percentage is shown	Pass
ST-04	Quick text analysis	Paste resume text and a job description	Match score shown, no file needed	Pass
ST-05	Unreadable PDF	Upload a scanned image PDF	Clear error message; nothing saved	Pass
ST-06	Dashboard and history	Analyse two resumes, then delete one	Counts and charts update correctly	Pass
ST-07	Job search and save	Search a keyword, save a job	Job appears in saved jobs	Pass
ST-08	Outside job listings	Admin refreshes the job feed	New jobs appear (needs internet)	Pass
ST-09	Employer approval	Register as employer; admin approves	Employer can now post jobs	Pass
ST-10	Job posting and approval	Employer posts a job; admin approves it	Job becomes visible to seekers	Pass
ST-11	Apply and rank	Two job seekers apply to one job	Employer sees both, ranked by score	Pass
ST-12	Status tracking	Employer shortlists an applicant	Job seeker sees the new status	Pass
ST-13	Closed job	Employer closes a job	Job disappears from search; can’t apply	Pass
ST-14	AI service unavailable	Stop the AI service, upload a resume	Friendly error message; nothing saved	Pass
ST-15	Access control	Try to open another user’s resume link	Access denied	Pass
ST-16	Expired login	Use the app after the login pass expires	User is sent back to the login page	Pass

4.3 Result Analysis

From end to end, the system works as planned: a job seeker signs up, uploads a resume, and gets back skills, a career guess, a score, missing skills, and suggestions; an employer

posts a job, the admin approves it, job seekers apply, and the employer sees applicants sorted by match score.

All the planned features were completed: secure login for three roles, PDF upload and text reading (for normal, non-scanned PDFs), skill extraction (120+ skills), career prediction (25 careers, rule-based plus a machine-learning model), score and match calculations, job, course, and interview-question suggestions, dashboard charts and history, external job importing, employer job posting with admin approval, ranked applications, and basic limits to stop abuse.

The machine-learning career model was trained on a mix of a public careers dataset, a hand-built skills table, and about 13,000 real, anonymised resumes, and checked on a held-out slice of the real resumes it had not seen during training. On that test slice, it reached about 86.1% accuracy and a macro-F1 score of about 0.83, which is a solid result for a first working version.

Overall, the system’s biggest strengths are its clear three-part design, its easy-to-explain skill-gap results, its reasonably wide career and skill coverage, and its automatic applicant ranking. Its biggest weaknesses are that it still only reads plain text PDFs (no scanned documents), its word-matching cannot yet understand similar words with different spelling, and a few features such as password reset are still missing.

CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS

5.1 Conclusion

This project built the AI Career Prediction System with an Integrated Job Portal. It reads PDF resumes, guesses a good career out of 25 options, finds missing skills, gives suggestions, and lets companies post jobs and see applicants automatically ranked by fit. The system was built in three parts (React, Laravel, and FastAPI), built step by step in small stages, and checked using unit and full-system tests.

Every goal set out in Chapter 1 was met: the system gives job seekers honest feedback, saves companies time when hiring, and puts career advice and job hunting in one place. The team also learned a great deal about natural language processing, machine learning, building APIs, secure logins, and full-stack web development along the way. The machine-learning model already performs well on real resume data, which is a strong base for the improvements listed below.

5.2 Future Recommendations

5.2.1 Smarter AI

- Grow the skill list beyond 120 skills using public skill lists such as ESCO or O*NET.
- Use smarter text-matching, such as sentence embeddings, so similar words are understood, not just exact matches.
- Add support for reading scanned resumes using OCR.
- Keep retraining the model as more real resumes become available, to keep accuracy improving over time.

5.2.2 Safety and Reliability

- Add password reset and email verification.
- Protect the AI service so only the main server can call it directly.
- Add more automated tests and regular health checks to the deployment pipeline.

5.2.3 Better Engineering

- Package the three services so they are easier to set up (for example, using Docker).
- Add caching to make the system faster under heavier use.
- Add paging to long lists of results.
- Refresh outside job listings automatically on a schedule instead of by hand.

5.2.4 New Features

- Add notifications for status changes and messages between employer and candidate.
- Let users download their resume analysis as a PDF, or import a resume from LinkedIn or GitHub.
- Add salary information, a dark mode, and a fuller admin dashboard.

REFERENCES

[1] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2016.

[2] “Poppler PDF rendering library.” freedesktop.org. [Online]. Available: https://poppler.freedesktop.org (accessed Sep. 20, 2026).

[3] “spaCy: Industrial-strength natural language processing in Python.” Explosion. [Online]. Available: https://spacy.io (accessed Sep. 20, 2026).

[4] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Inf. Process. Manag.*, vol. 24, no. 5, pp. 513–523, 1988.

[5] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, Oct. 2011.

[6] “React documentation.” Meta Platforms. [Online]. Available: https://react.dev (accessed Sep. 20, 2026).

[7] “Laravel documentation.” Laravel. [Online]. Available: https://laravel.com/docs (accessed Sep. 20, 2026).

[8] “FastAPI.” [Online]. Available: https://fastapi.tiangolo.com (accessed Sep. 20, 2026).

[9] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Eng. Task Force, RFC 7519, May 2015.

[10] “LinkedIn Jobs.” LinkedIn Corp. [Online]. Available: https://www.linkedin.com/jobs/ (accessed Sep. 20, 2026).

[11] “Indeed.” Indeed, Inc. [Online]. Available: https://www.indeed.com (accessed Sep. 20, 2026).

[12] “Merojob.” [Online]. Available: https://merojob.com (accessed Sep. 20, 2026).

[13] “Jobscan.” [Online]. Available: https://www.jobscan.co (accessed Sep. 20, 2026).

[14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, Minneapolis, MN, USA, 2019, pp. 4171–4186.

[15] European Commission, “ESCO: European skills, competences, qualifications and occupations.” [Online]. Available: https://esco.ec.europa.eu (accessed Sep. 20, 2026).

[16] National Center for O*NET Development, “O*NET OnLine.” [Online]. Available: https://www.onetonline.org (accessed Sep. 20, 2026).

[17] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*, 2nd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2005.

BIBLIOGRAPHY

[1] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill, 2019.

[2] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2019.

[3] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. draft, 2023. [Online]. Available: https://web.stanford.edu/~jurafsky/slp3/

APPENDIX A: SCREENSHOTS

This appendix holds screenshots of the system's main screens.

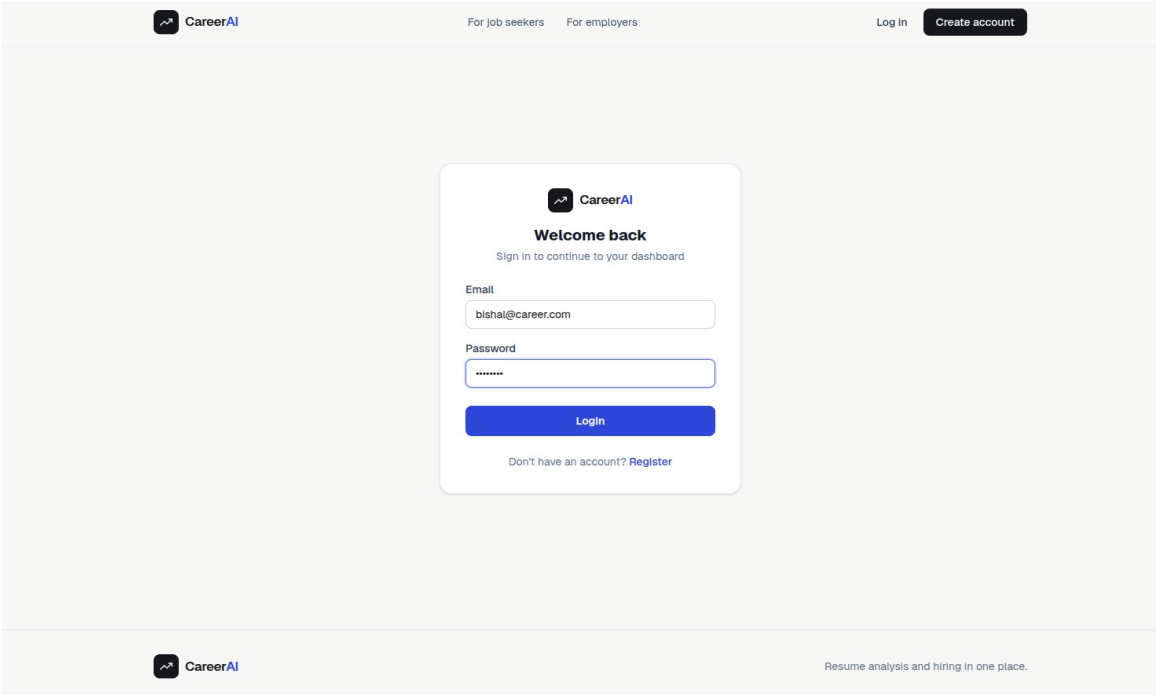


Figure 16: Login/Signup Page and Job Seeker Dashboard

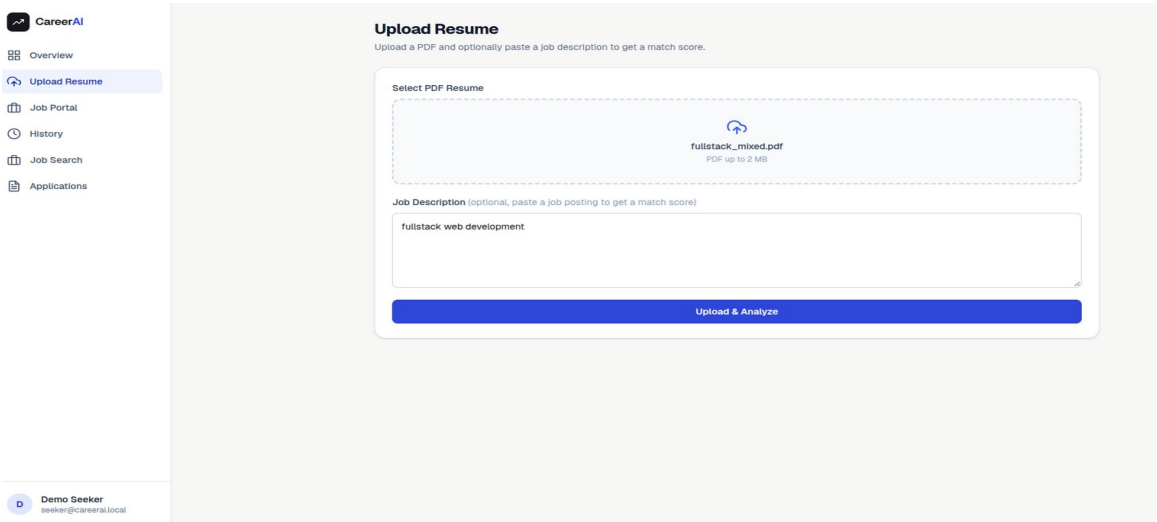


Figure 17: Resume Upload Page and Analysis Result Page

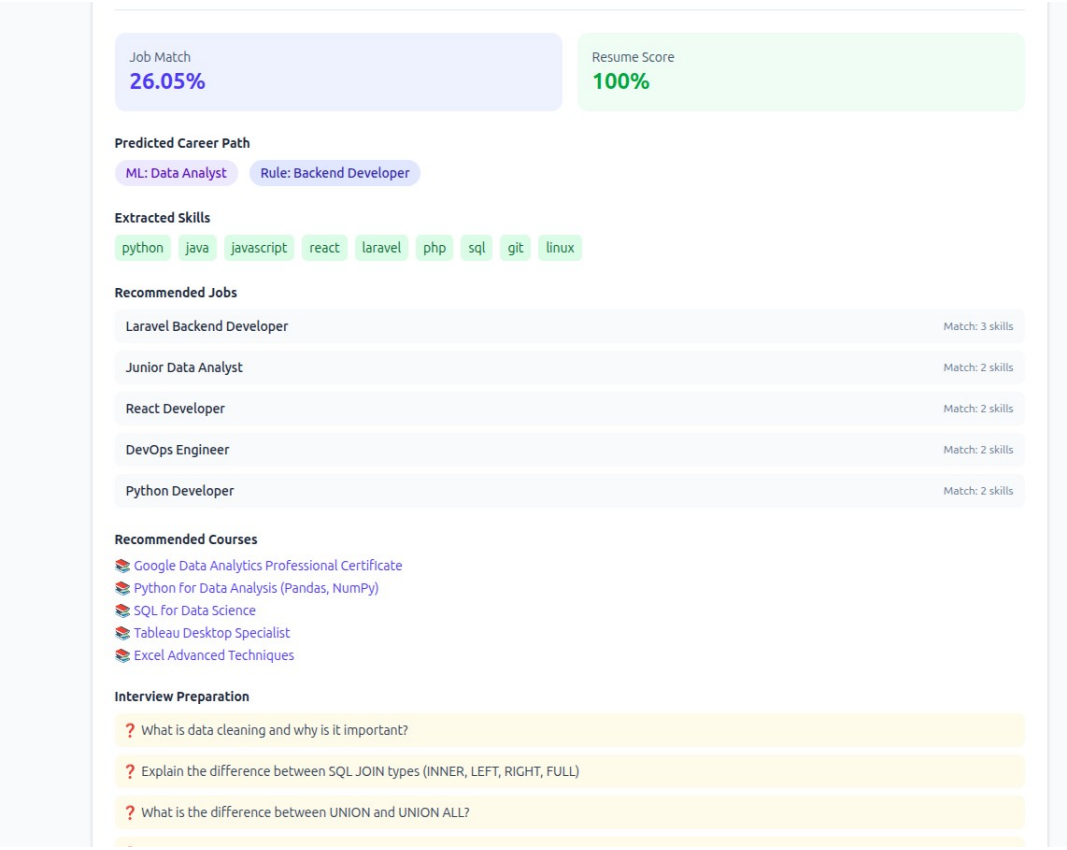


Figure 18: Skill Gaps and Analyze Page

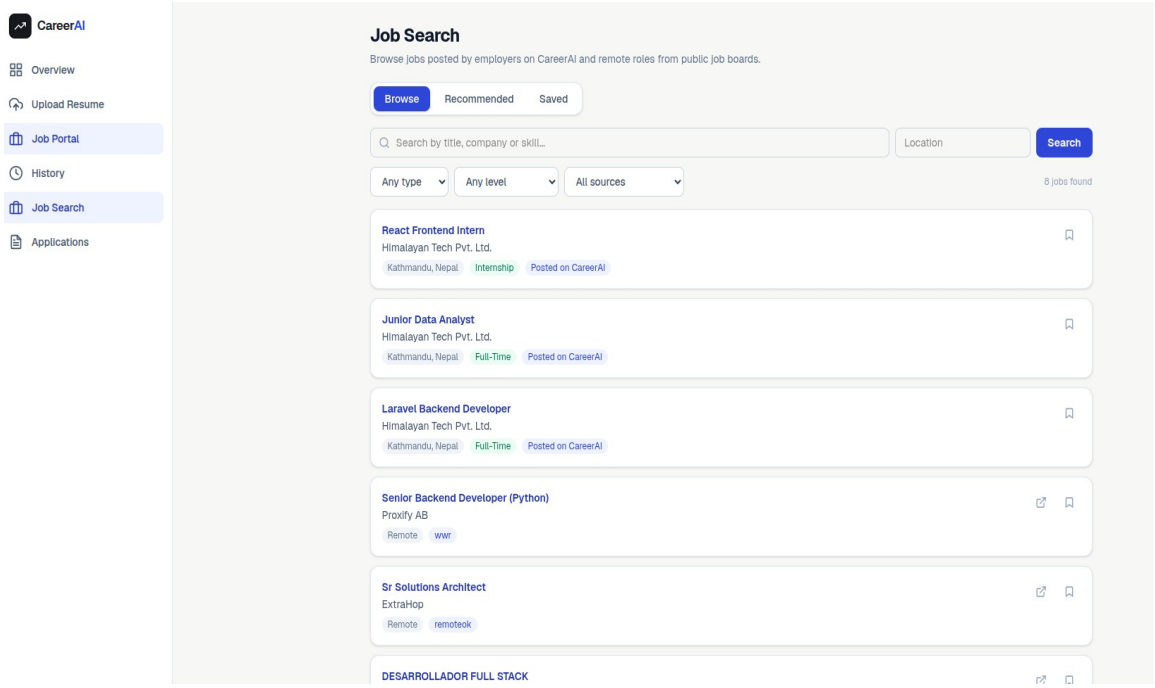


Figure 19: Job Details/Apply Page and My Applications Page

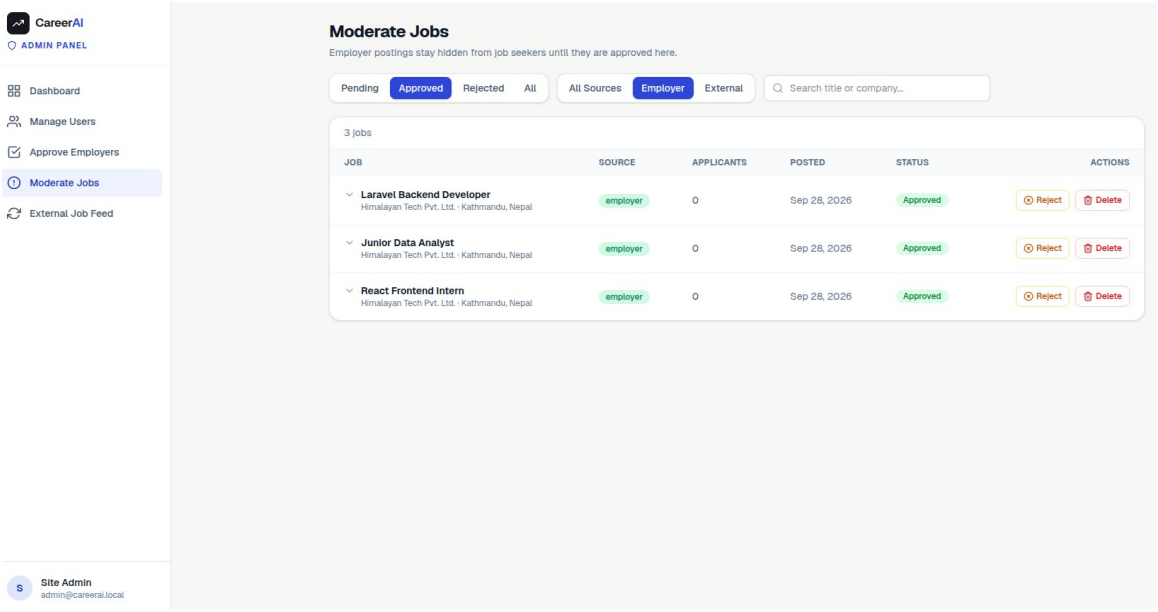


Figure 20: Employer Job Posting Page and Admin Moderation Page